ASSIGNMENT APPROACH DOCUMENT

AI Engineering Internship — Tri9T AI

Project: CardioTrack CT-200 Manual — Structured Extraction, Versioning & QA Test Generation

Prepared by: Jayanthi K

Document status: Technical approach draft — validate against the supplied CT-200 v1/v2 manuals and actual implementation before submission.

Purpose

This document outlines a practical design for an API that extracts the CT-200 manual into a hierarchical, versioned document tree; pins QA selections to exact versions; generates structured test-case ideas using an LLM; and exposes traceability and staleness information.

> Important accuracy note: the assignment brief was available for this draft, but the separate CT-200 manual data package (including v1 and v2) and implementation/debugging results were not provided with it. Therefore, this document does not claim specific irregularities were observed or that particular bugs were already fixed. Complete the validation notes with evidence from your actual run before submitting.

1. Problem Understanding

The system must convert the supplied medical-device manual into a browsable tree while preserving headings, section levels, body text, parent/child relationships, tables and figure captions where possible. It must retain historical versions, identify changed content, generate 3–5 QA test-case ideas from a version-pinned selection, and make the validity of previously generated cases queryable after re-ingestion.

The key engineering risk is not merely extraction failure. A parser can return valid-looking data while attaching text to the wrong section or losing a requirement. For that reason, the design prioritizes traceability, explicit uncertainty, repeatable tests, and validation against the source pages.

2. Proposed Architecture and Technology

Component	Proposed choice	Purpose
API	FastAPI + Pydantic	REST endpoints and request/response validation.
PDF extraction	PyMuPDF/pdfplumber for text and layout; OCR (for example Tesseract) only where page text is absent or image-based.	Use native text when reliable; OCR scanned pages and preserve page references.
Persistence	SQLAlchemy + SQLite	Document versions, nodes, logical-node mappings, selections and provenance.
Generated outputs	JSON store or MongoDB, with a documented choice	Persist validated LLM output with selection and exact source-node/version references.
LLM integration	Provider-agnostic client with structured JSON output and validation	Generate test ideas while handling malformed or incomplete responses.
Testing	pytest + fixtures + API tests	Verify irregular parsing cases, version matching and stale-status behavior.

This stack follows the assignment's suggested backend stack. If a different OCR library or generated-output store is selected during implementation, the final document should name the actual choice and justify it.

3. Parsing and Hierarchy Reconstruction

3.1 Inspection before implementation

First inspect the full CT-200 v1 and v2 PDFs page by page. Record page count, whether text is selectable, heading numbering patterns, font/position cues, multi-column regions, tables, figures, repeated headers/footers, and any inconsistent section formatting. Do not infer that the entire manual follows the first few pages.

3.2 Extraction pipeline

1. Detect whether each page has usable embedded text. Extract native text and layout when available; apply OCR to scanned/image-only regions. Keep page number and bounding-box/source-span metadata for traceability.
2. Normalize whitespace and line breaks conservatively. Do not remove numbering, units, symbols, warning labels, or thresholds as generic cleanup.
3. Identify heading candidates using numbering patterns, typography/layout signals, and document-specific rules. Use explicit rules for known CT-200 patterns rather than claiming universal PDF support.
4. Build a hierarchy with a stack of the most recent headings by level. A new heading attaches to the nearest preceding valid parent at the expected level. If the level or parent is ambiguous, flag it for review instead of silently guessing.

5. Attach paragraphs and lists to their current section. Represent tables and figures as typed content blocks where extraction is reliable; preserve captions and page references. Record unsupported or low-confidence regions explicitly.

3.3 Node model

Each node should include: node ID; document ID; version ID; heading/title; level; body/content blocks; parent ID; ordered child IDs; page/source references; content hash; logical-node ID across versions; extraction confidence or review flag. Use a deterministic content hash (for example SHA-256 over canonicalized heading and body content) to detect exact content changes.

4. Version Matching and Staleness

Store every ingestion as a new immutable document version. Do not overwrite version 1. For each new node, match to a prior logical node using a combination of normalized heading/path, parent context, and body similarity. Exact path/title matches are useful but insufficient when sections are renamed or duplicated; content hash alone cannot match nodes whose body changed. Preserve a match rationale and an ambiguity flag where multiple candidates score similarly.

Recommended change statuses include **unchanged**, **content_changed**, **moved_or_renamed**, **added**, **removed**, and **ambiguous_match**. A selection stores explicit (node ID, version ID, content hash) references so it continues to resolve to the original text after re-ingestion.

At retrieval time, compare the exact source node/version/hash attached to each generated output with the latest matched logical node. Return a stale status and a lightweight diff summary when content differs. Textual changes alone cannot reliably judge clinical significance: a one-word change may be harmless or may change a safety requirement. For high-risk numeric limits, units, timing, warnings and negations, add targeted comparisons and flag the result for human review.

5. API Plan

Capability	Illustrative endpoint	Expected behavior
List sections	GET /documents/{id}/sections?version={v}	Return root-level sections for requested or latest version.
Get node	GET /nodes/{node_id}?version={v}	Return heading, full content, hash, children and source references.
Search	GET /search?q=...&version;={v}	Search heading/body with version filtering and stable result IDs.
Compare versions	GET /nodes/{logical_id}/changes?from =v1&to;=v2	Return change status and concise text diff.
Create selection	POST /selections	Save named selection with exact node/version/hash references.
Generate QA ideas	POST /generations	Reconstruct pinned source text, call LLM, validate 3–5 test cases, and persist provenance.
Retrieve generations	GET /generations?selection_id=... or node_id=...	Return output, source references and current staleness status.

Endpoint names are proposed examples, not a claim that these routes have already been implemented. The README and final document must match the actual repository.

6. LLM Prompting, Validation and Duplicate Requests

Provide the model with only the selected, version-pinned text and instruct it to return 3–5 concrete QA test ideas. Each item should include a title, requirement/source reference, preconditions, steps, expected result, and risk or priority when supported by the source. The model must not invent thresholds or device behavior absent from the manual; unsupported assumptions should be identified explicitly.

Validate the response against a Pydantic schema, including required fields, list length (3–5), non-empty steps, and source traceability. If JSON parsing or schema validation fails, retry once with a correction prompt that includes validation errors. If the retry fails, return a clear generation failure and store diagnostic metadata rather than presenting malformed output as valid. Keep secrets in environment variables and avoid logging API keys.

For duplicate submissions, use an idempotency key or a canonical fingerprint of selection ID, pinned node hashes, prompt version and model settings. The proposed policy is to return the existing generation for an identical request unless the caller explicitly requests regeneration; a forced regeneration creates a separate generation record. This avoids accidental duplicates while preserving an audit trail.

7. Validation and Test Strategy

At minimum, create explicit unit tests for three real irregularities found in the CT-200 PDF. Candidate test categories (confirm against the actual manual before treating them as observed) are: (1) duplicate headings remain distinct nodes with correct parents; (2) a subsection with unusual numbering/formatting attaches to the correct parent; (3) table or figure caption content remains associated with the right section; (4) repeated page headers/footers are not merged into body text. Add tests for unchanged/changed nodes across v1→v2, version-pinned selections, malformed LLM output, repeated generation requests, and stale-status retrieval.

Validation should combine automated assertions with manual source comparison. For each page, compare extracted text and hierarchy against the rendered PDF. Check that all detected headings have one parent (except document

root), child order follows source order, text is not silently dropped, and ambiguous regions are surfaced. Run an end-to-end demonstration: ingest v1, create selection, generate test ideas, ingest v2, then retrieve the old generation and show its staleness status.

8. Observed Edge Cases and Implementation Changes — Evidence Required

> This section must be completed after running the parser on the supplied CT-200 v1/v2 manuals. The assignment brief explicitly asks for observed structural inconsistencies, what the initial implementation failed to handle, how failures were detected, and what changed. Those facts cannot be established from the assignment brief alone.

Observed PDF irregularities: Record the exact page/section, what was inconsistent, and the expected versus extracted result. Do not list candidate examples above as findings unless confirmed by the source.

Initial implementation failure: Describe a reproducible example, such as a wrong parent, missing paragraph, broken table, or stale-match error, only if it actually occurred.

Detection method: Name the manual inspection, unit test, validation script, or visual comparison that exposed the issue, and include its result.

Correction made: Explain the code/rule change and the before/after outcome. Link each correction to a test that prevents regression.

9. Decision Log

1. What is most likely to silently give wrong results?
The highest-risk silent failure is incorrect hierarchy or version matching: content can look plausible while being attached to the wrong parent or linked to the wrong logical node. I would catch it with page-level source references, hierarchy invariants, duplicate-heading and parent-assignment tests, and manual comparison against the rendered PDF. Ambiguous matches should be flagged instead of automatically accepted.

2. Where did I choose simplicity over correctness, and what would break first?
For an initial implementation, I would use CT-200-specific heading rules and a lightweight similarity matcher rather than a generic document-understanding system. This is a deliberate scope choice, not a claim of universal PDF support. The first weaknesses would appear with unseen layouts, major section reordering, similar repeated headings, or changes where a small textual edit has a large safety impact.

3. Name one input not handled and the system response.
A page that is severely blurred or has a complex table whose reading order cannot be reconstructed reliably is an explicit unsupported case. The pipeline should mark the page/block as low-confidence or requiring review, retain the page reference, and avoid silently inventing a hierarchy. The final implementation should document the actual unsupported case encountered or tested.

10. Known Limitations and Next Steps

This approach uses document-specific rules and a practical matching heuristic, so it will not reliably generalize to arbitrary manuals. OCR can misread units, punctuation, subscripts, and numbers; fuzzy matching can confuse repeated headings; and a text diff cannot determine clinical significance by itself. Before production use, I would add confidence calibration, richer layout/table extraction, human review for safety-critical changes, stronger semantic matching evaluation, and broader regression fixtures.

> Before submission, verify that the repository, endpoint names, actual parser/OCR libraries, database choices, test results, observed PDF irregularities, and implementation changes described here match your real code and evidence. Add the GitHub repository URL and ensure the README demonstrates the complete v1→v2 staleness flow.

Reference basis

Prepared from the Tri9T AI Engineering Internship Assignment brief. Requirements reflected include hierarchical PDF extraction, versioned ingestion, browse/search APIs, version-pinned selections, LLM-generated QA ideas, staleness detection, retrieval, tests, README, Git history, and the required decision log.